Real-Time Network Telemetry Framework: Collecting, Streaming and Visualizing Bandwidth and Latency of Routers and Switches

Avinash Reddy
23110123

Gella Jayarama Krishna
23110115

Sowpati Raj Kamal
23110319

Jangam Sanjay
23110144

Pulakurthi Manohar
23110259

Yalla Sai Teja
23110366

Abstract—*We present a modular network telemetry framework that collects bandwidth, utilization, latency and packet-loss metrics from a laptop and simulated network devices. The collected measurements are transmitted to a central backend through persistent gRPC streams using Protocol Buffers. The backend validates and stores telemetry in SQLite, evaluates threshold-based alerts and provides a real-time dashboard, REST APIs for historical data and CSV/JSON exports. Prometheus metrics are also exposed for monitoring integration. The framework demonstrates a lightweight approach to collecting, processing and visualizing network performance metrics without requiring physical routers or switches.*

I. Introduction

Network operators must know, at any moment, which links are close to saturation, where delay is building up and whether packets are being lost. Classic monitoring polls devices with SNMP every few minutes; such coarse averages hide the short congestion bursts that users actually feel, and every new device has to be configured on the central poller. Network telemetry, as framed by RFC 9232 [1], instead treats monitoring data as a continuous stream that is generated close to the source, exported in a normalized form and consumed by many applications.

This project builds a lightweight telemetry pipeline for monitoring network performance using laptop-based and simulated network-device agents. The laptop collector reads operating-system network statistics using `psutil`, while simulated agents generate router and switch telemetry. Both sources produce normalized Metric messages and transmit them over persistent gRPC streams using Protocol Buffers to a central backend. The backend validates and stores measurements, evaluates threshold-based alerts, and serves a live dashboard, REST APIs for historical data and export, and a Prometheus metrics endpoint.

Our main contributions are:

1. A modular telemetry pipeline covering collection, transmission, storage, alerting, visualization and data export.
2. Laptop-based and simulated network-device agents that produce normalized telemetry measurements.
3. A gRPC and Protocol Buffers communication layer with backend ingestion, SQLite storage and threshold-based alert management.
4. A web-based dashboard and REST API providing real-time monitoring, historical data access and CSV/JSON export, with Prometheus monitoring integration.

II. Problem Statement

Problem. Given a set of network devices D, each exposing interface counters (octets, packets, discards, errors) and a link speed, produce for every device d and every interval k of length Δ (default 2 s) a sample m(d,k) = (download bit/s, upload bit/s, capacity, utilization, latency, loss), deliver it to one central store with bounded delay, raise an alert whenever a metric stays above a configured threshold, and make the full history queryable and exportable.

	Definition
Inputs	Laptop or simulated-device agent configuration; interval Δ; globally configured warning and critical thresholds per metric.
Outputs	Live dashboard; alert list (open/resolved, severity, peak value, duration); REST time series; CSV/JSON export; Prometheus metrics.
Assumptions	Laptop-based and simulated network-device agents are available; agent and backend clocks are sufficiently synchronized; one backend host.

	Definition
Constraints	Telemetry must not disturb the network it measures (RFC 9232 §2.4); new devices must appear without backend configuration; agents and backend must survive each other's restarts; must run on a laptop without special hardware.
In scope	Management plane telemetry of interface statistics (RFC 9232 §3.1.1); threshold alerting; history and export.
Out of scope	Control plane (e.g. BMP) and forwarding plane (IPFIX, IOAM) telemetry; per-flow analysis; SNMPv3; high-availability clustering.

Motivating scenario. A branch link with 30 % background load saturates to 95 % for 40 s. A 5-minute poller reports $(40 \cdot 95 + 260 \cdot 30)/300 \approx 38.7$ %, well below a 70 % warning level, so the event is invisible; with 2 s sampling the same burst produces 20 samples above 90 % and a critical alert within seconds.

Gap. Periodic SNMP polling is universally supported but coarse and centrally configured; model-driven streaming telemetry such as gNMI [8] is fine-grained but requires recent devices that support it; full observability stacks are heavy to deploy for a small network. This project demonstrates push-based streaming telemetry, a normalized data model and threshold-based alerting using laptop-based and simulated network-device agents.

III. Project Objectives

#	Objective	Target (measured in Section VII)
O1	Collect rx/tx bandwidth, utilization, latency and loss from every device at a fixed interval	$\Delta = 2$ s (configurable), all five metrics
O2	Deliver samples to the backend in near real time	p95 end-to-end delay < 1 s with ≥ 100 devices
O3	Detect sustained threshold breaches quickly and clear them automatically	Alert within 2 intervals of the breach; auto-resolve after recovery
O4	Avoid false alarms from single noisy samples	Alerts ≈ real episodes (no flapping)
O5	Historical access and export through REST	24 h of 10 devices queried in < 1 s; CSV/JSON export
O6	Support multiple simulated network devices and laptop telemetry collection through a common backend.	Hundreds of devices on one laptop with no samples dropped
O7	Provide efficient telemetry transmission using gRPC and Protocol Buffers.	< 100 B per sample on the wire
O8	Support reliable agent communication through local buffering and reconnection with exponential backoff.	No data loss across backend restarts (agent buffer); no duplicate alerts

IV. Architecture and Design

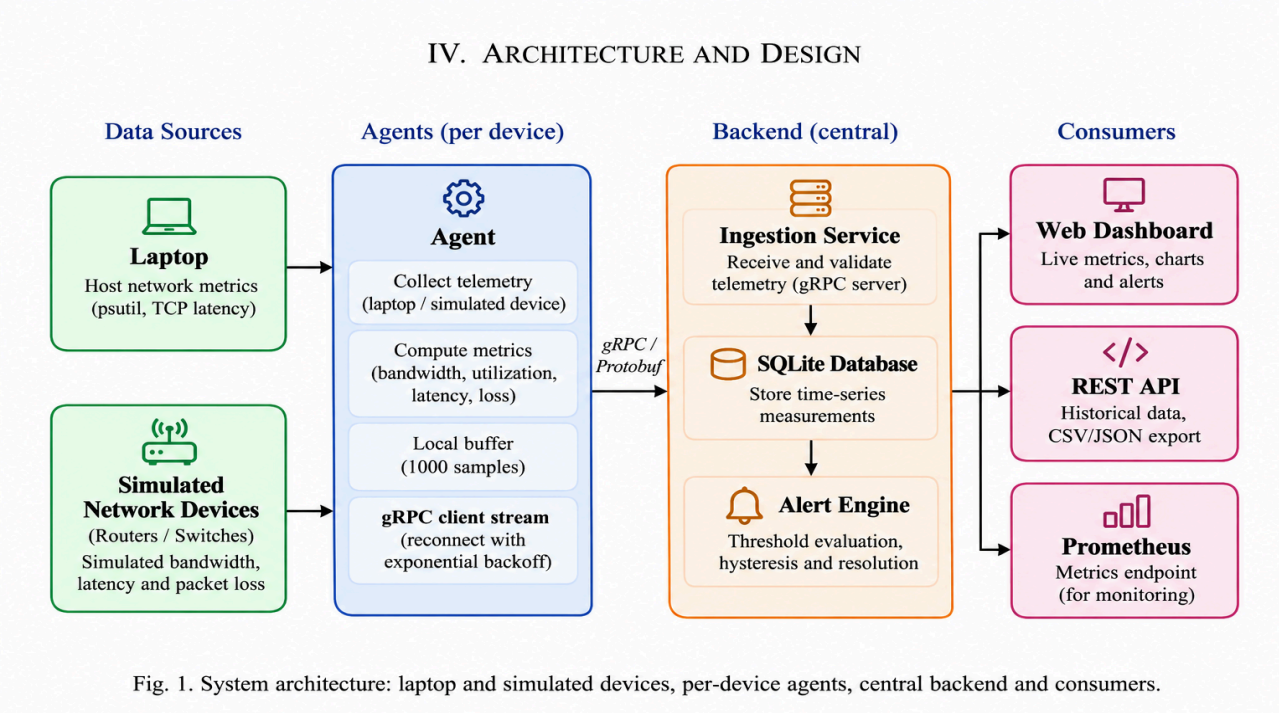


Fig. 1. System architecture: laptop and simulated devices, per-device agents, central backend and consumers.

Fig. 1. *System architecture: laptop and simulated devices, per-device agents, central backend and consumers.*

A. Components

- **Collectors** produce one Metric per interval. The laptop collector reads operating-system network counters using psutil, while simulated collectors generate network traffic, utilization, latency, congestion and packet-loss scenarios.
- **Agent.** Two threads decouple measuring from sending: the measure loop puts each sample in a local buffer (1000 samples, oldest dropped first), the send loop drains it into one long-lived gRPC client stream (`Telemetry.Stream`) and reconnects with exponential back-off (1, 2, 4 … 10 s) if the backend is unreachable.
- **Transport.** proto/telemetry.proto defines the Metric message containing device information, interface details, timestamps, bandwidth, capacity, latency and packet-loss measurements. Agents send telemetry data to the backend through a persistent gRPC streaming connection using Protocol Buffers.
- **gRPC receiver and ingestion.** The backend receives incoming telemetry samples, validates them and places them in an ingestion queue. A background writer stores samples in SQLite using batched transactions, reducing database write overhead.
- **Alert engine.** The backend monitors incoming metrics against configured thresholds. It uses a hysteresis mechanism, requiring two consecutive threshold breaches to open an alert and three healthy samples to resolve it. A monitoring process also detects devices that stop sending telemetry.
- **Consumers.** The system provides a live web dashboard, a FastAPI REST API for historical data and CSV/JSON exports, and a Prometheus metrics endpoint that can be integrated with Grafana**.**

B. Mapping to RFC 9232

The project follows the Management Plane Telemetry model described in RFC 9232 (§3.1.1). Its architecture incorporates the key functional components described in §3.2: Data Object and Source (host operating-system counters and simulated device metrics), Data Generation and Processing (metric calculation and validation), Data Encoding and Export (Protocol Buffers over gRPC), Data Configuration and Subscription (agent configuration and collection intervals), and Data Query, Analysis and Storage (database storage, alert processing, dashboard and REST APIs). Telemetry is collected by agents and pushed to the central backend through a persistent gRPC stream.

C. Design Decisions and Trade-offs

TABLE I
KEY DESIGN DECISIONS

Decision	Alternatives considered	Why this choice	Trade-off
Push over a gRPC client stream	REST POST per sample; MQTT broker; Kafka	One HTTP/2 connection per device, typed contract, the same push style as gNMI	Needs gRPC libraries; harder to inspect by hand than JSON
Protobuf encoding	JSON	72 B per sample vs 228 B as JSON (Section VII-B)	Not human-readable on the wire
SQLite + batched writer	InfluxDB, TimescaleDB, Prometheus TSDB	Zero setup, one file; all SQL isolated in `backend/db.py`	Single node; limited write rate
Queue between receive and store	Write inside the gRPC handler	Receiving never waits for the disk; batches amortize commits	Overload drops samples (counted, visible in /api/health)
Hysteresis: 2 breaches open, 3 healthy close	Alert on every breach	Suppresses flapping (Section VII-D)	One extra interval of detection delay
Own dashboard + optional Grafana	Grafana only	Works offline without Docker	Less flexible than Grafana

V. IMPLEMENTATION

The system is written in Python 3.12 (≈2 500 lines of application code, 815 lines of tests) plus a 494-line HTML/JavaScript dashboard. Table II lists the main building blocks.

TABLE II
LANGUAGES, FRAMEWORKS AND LIBRARIES

Layer	Technology
Agents	Python threads, gRPC client, psutil for host operating-system counters
Transport	gRPC over HTTP/2, Protocol Buffers
Backend	Python, gRPC server, SQLite, FastAPI, Uvicorn
Front end	HTML, CSS, JavaScript, canvas-based dashboard
Deployment	Docker Compose, Prometheus and Grafana
Testing	pytest, FastAPI TestClient, unit and integration tests

A. Telemetry Collection

The framework collects telemetry from two sources:

1. Laptop Agent: Uses `psutil` to read operating-system network interface counters, including received and transmitted bytes, packets, errors and drops. Bandwidth is calculated from counter differences over the collection interval. Latency is measured using TCP connection time to public DNS servers.

2. Simulated Device Agent: Generates telemetry for simulated routers and switches, including bandwidth, utilization, latency and packet-loss metrics.

Both agents convert the collected measurements into a common `Metric` message and transmit them to the backend through a persistent gRPC stream using Protocol Buffers.

B. Alert state machine (per device and metric)

```
Alert Generation and Resolution

The alert engine continuously compares incoming telemetry measurements against predefined
warning and critical thresholds. It uses a hysteresis-based mechanism to reduce false alerts
caused by temporary fluctuations.

    • Alert opening: An alert is generated when a metric breaches its configured threshold
      for two consecutive collection intervals.
    • Severity updates: If an active alert continues to breach the threshold, its severity
      and peak observed value are updated.
    • Alert resolution: An active alert is resolved after the metric remains within the
      healthy range for three consecutive intervals.
    • Device offline detection: If no telemetry is received from a device for more than 10
      seconds, the system generates a device-offline alert.

The thresholds are globally configured and applied across devices.
```

C. Challenges and Solutions

- Telemetry source differences: Laptop and simulated-device agents produce different measurements. We standardized them using a common `Metric` message.
- Unavailable measurements: Some measurements, such as latency, may be unavailable when a connection attempt fails. The collector handles these cases without treating missing values as valid measurements.
- Reliable transmission: Temporary backend disconnections can interrupt telemetry delivery. Agents use local buffering and exponential-backoff reconnection.
- Efficient database ingestion: Writing every measurement individually can increase database overhead. A queue and background batch writer are used for SQLite ingestion.
- False alert prevention: Temporary metric fluctuations can trigger unnecessary alerts. The alert engine uses consecutive-breach and consecutive-healthy checks.

Code repository: https://github.com/Rajkamal065/network-telemetry

VI. Testbed Setup

The testbed consists of a laptop-based environment containing simulated network-device agents, a laptop telemetry collector, and a central backend server. The agents collect or generate network telemetry and transmit measurements to the backend through persistent gRPC streams using Protocol Buffers. The backend processes incoming telemetry, stores measurements in SQLite, evaluates alert conditions, and exposes data through REST APIs and a real-time dashboard. No physical routers or switches are required for the current implementation.

TABLE III
Testbed

Item	Configuration
Hardware	Dell laptop, Intel Core i5-12500H (12 cores / 16 threads), 8 GB RAM (7.7 GB usable)
OS / runtime	Windows 11 Home (build 26200); Python 3.12.10; grpcio 1.85.1; protobuf 7.36.2; FastAPI 0.162.0; uvicorn 0.54.0; SQLite 3.49.1
Topology	Simulated network-device agents, laptop collector, and central backend communicating over gRPC.

Item	Configuration
Workloads	Simulated network traffic, bandwidth, latency and packet-loss scenarios, alongside live laptop network measurements.
Parameters	Interval 2 s (1 s in E1); open after 2 breaches, close after 3 healthy samples; offline after 10 s; thresholds utilization 70/90 %, latency 80/150 ms, loss 2/5 %; ingest queue 20 000, batch 500; gRPC workers 512 in E1

TABLE IV
EXPERIMENTS

#	Experiment	Method
E1	Scalability	5–400 simulated devices at 1 sample/s; 10 s warm-up, 40 s measurement; stored rate, drops, CPU, memory, end-to-end delay
E2	Message overhead	Serialized size of one telemetry sample: Protocol Buffers vs equivalent JSON representation.
E3	Alert detection	Six 20 s congestion incidents on one switch at $\Delta = 2$ s; wall-clock time to alert open and to resolve
E4	Flapping	Trace-driven replay: 1 h utilization traces (wave + noise, 20 seeds) with a 50 % warning level, three hysteresis settings
E5	REST API	24 h of 10 devices at 2 s (432 000 rows); median of 7 requests per endpoint

VII. EVALUATION

A. Scalability and End-to-End Delay (E1)

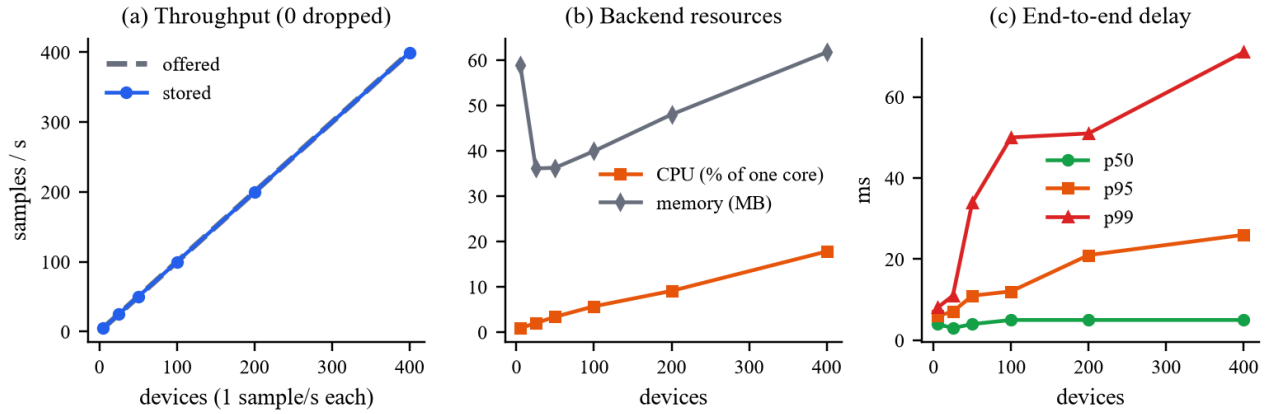


Fig. 2. E1: (a) samples offered vs stored per second; (b) backend CPU and memory; (c) delay from measurement at the agent to stored and alert-checked at the backend.

Fig. 2(a) shows that the backend stored exactly the offered load at every size up to 400 devices (1 sample per device per second, 400 samples/s); across all runs 0 samples were dropped or rejected. CPU use of the backend grows roughly linearly with the sample rate, reaching 17.8 % of one core at 400 devices, and its memory stayed below 62 MB, so a single laptop core is far from saturated (Fig. 2(b)). Fig. 2(c) shows the end-to-end delay (agent timestamp to stored and alert-checked): the median stays at 3–5 ms and the 95th percentile at or below 26 ms. The 99th percentile rises with load (8 ms at 5 devices, 71 ms at 400) because samples queue behind batched database commits; this is the price of batching, and it remains well inside one sampling interval. Since load depends on samples per second, 400 samples/s corresponds to 800 devices at the default $\Delta = 2$ s.

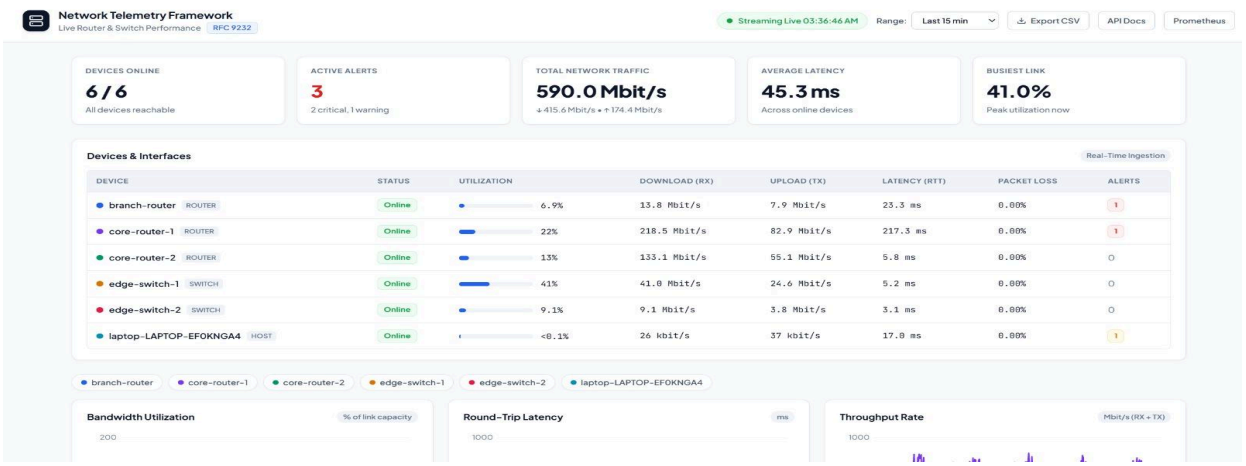


Fig. 3. *Real-time network telemetry dashboard showing device status, active alerts, network traffic, latency and interface-level metrics.*

The dashboard provides a consolidated view of all connected devices, displaying their network utilization, received and transmitted bandwidth, round-trip latency and packet loss. It also presents active alerts and overall network statistics.

TABLE V
E1 RESULTS (1 SAMPLE PER DEVICE PER SECOND)

Devices	Offered /s	Stored /s	Dropped	CPU (% core)	RSS (MB)	p50 (ms)	p95 (ms)	p99 (ms)
5	5	5.0	0	0.9	59	4	6	8
25	25	25.0	0	2.0	36	3	7	11
50	50	50.0	0	3.4	36	4	11	34
100	100	99.9	0	5.7	40	5	12	50
200	200	199.9	0	9.1	48	5	21	51
400	400	399.8	0	17.8	62	5	26	71

B. Telemetry Overhead (E2)

One sample is 72 bytes as protobuf and 228 bytes as equivalent JSON, a 3.2× saving. At $\Delta = 2$ s a device therefore costs 288 bit/s of payload towards the backend (plus HTTP/2 framing on the shared stream), i.e. about 0.29 Mbit/s for 1000 devices.

C. Alert Detection and Recovery (E3)

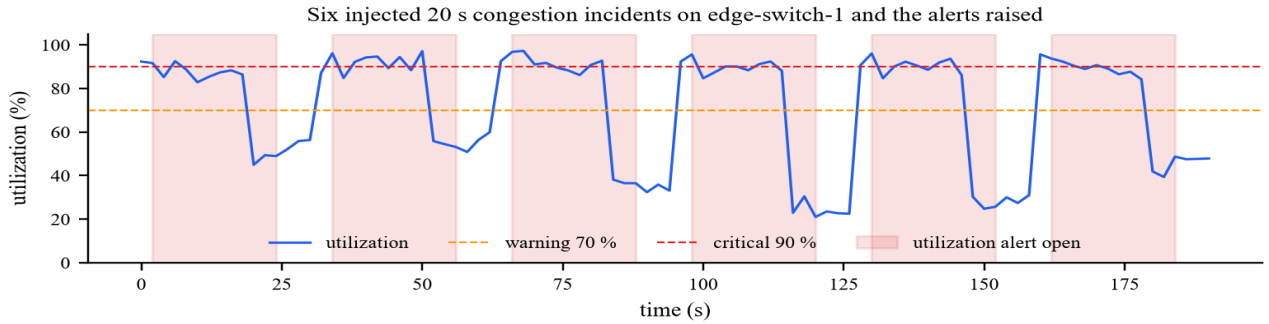


Fig. 4. E3: utilization of edge-switch-1 during six injected congestion incidents; shaded spans are the open utilization alerts.

All 6 of 6 incidents raised a utilization alert, on average 2.8 s after the congestion began (min 2.1 s, max 3.9 s), and every alert resolved by itself on average 4.8 s after the incident ended. This matches the design: the first congested sample arrives up to one interval (2 s) after onset and the second breach opens the alert, so detection must lie between 2 and 4 s; clearing needs three healthy samples, i.e. between 4 and 6 s after the end. The congestion also raised latency (queueing delay) and packet-loss alerts; all 18 alerts recorded during the run lie inside the six incident windows, so no alert was raised between incidents.

D. Effect of Hysteresis on False Alerts (E4)

TABLE VI
E4: ALERTS ON NOISY 1 H TRACES FOR THREE HYSTERESIS SETTINGS

Open after / close after	Alerts per hour (mean, 20 traces)	Worst trace	Real episodes per hour	Delay to alert a real incident
1 breach(es) / 1 healthy	101.1	114	29.1	2.0 s
2 breach(es) / 3 healthy	29.6	37	29.1	3.4 s
3 breach(es) / 5 healthy	27.6	34	29.1	4.7 s

The traces oscillate around the warning level, as a busy link does near its daily peak. Alerting on every breach (1/1) produced 101.1 alerts per hour although the signal crossed the threshold only 29.1 times: each crossing flapped into 3.5 alerts on average. The chosen setting (2/3) produced 29.6 alerts per hour, essentially one per real episode, a 71 % reduction, at the cost of 1.4 s more detection delay. A stricter 3/5 setting merges some genuine episodes (27.6 alerts for 29.1 episodes) and delays alerts further, so 2/3 is the best trade-off. (Detection delays here are below 2Δ when an open warning escalates to critical on the first congested sample.)

E. Historical Access through the REST API

TABLE VII
REST API WITH 432 000 STORED SAMPLES (24 H, 10 DEVICES, 49 MB DATABASE)

Request	Median latency	Response size	Points returned
GET /api/devices	95 ms	3.7 KB	-
GET /api/summary	94 ms	0.2 KB	-
GET /api/metrics (15 min, raw)	133 ms	620.9 KB	4440
GET /api/metrics (24 h, downsampled)	985 ms	842.9 KB	6010
GET /api/metrics/export CSV (1 h)	543 ms	2.0 MB	-
GET /metrics (Prometheus)	86 ms	4.6 KB	-

A full day of history for all devices is returned in 985 ms (slowest of 21 repetitions 1134 ms) because the server averages it into at most 600 points per device inside SQLite; this only just meets the 1 s target, and a pre-aggregated roll-up table would be the next step for longer ranges. A raw 15-minute window takes 133 ms, and the dashboard's per-refresh calls finish in about 90–130 ms each, well within its 2 s refresh period. In a paired measurement on the same database, the CSV export of one hour (18 000 rows) took 7.9 s with one network write per row (the first version) and 0.57 s with 1000-row chunks, a 14× improvement.

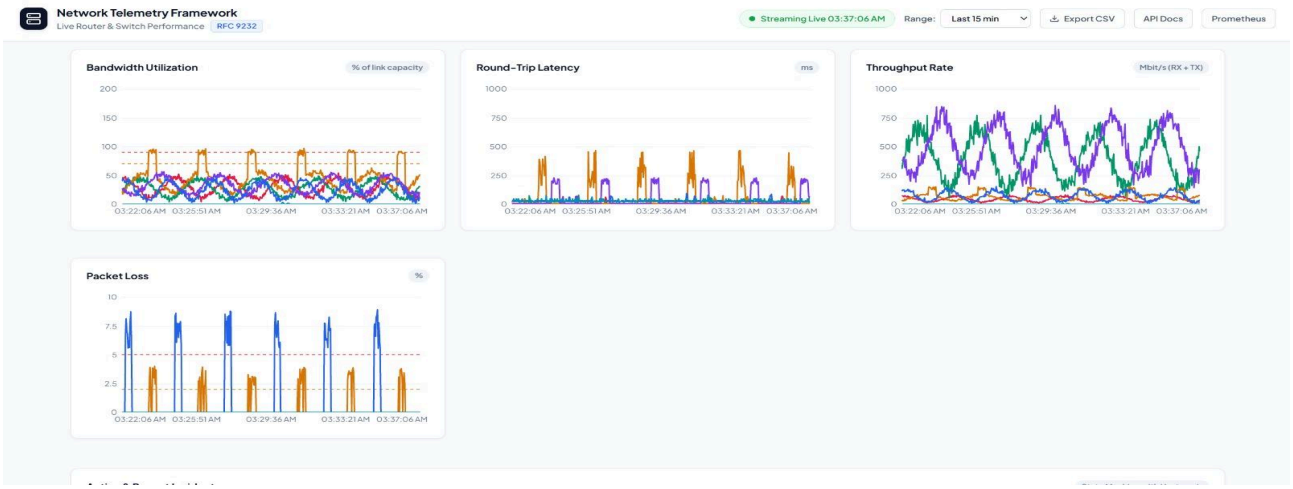


Fig. 5. *Time-series visualization of bandwidth utilization, round-trip latency, throughput rate and packet loss across monitored devices.*

The time-series charts illustrate changes in network performance over time. Separate visualizations allow users to observe bandwidth utilization, latency, throughput and packet-loss variations across different devices.

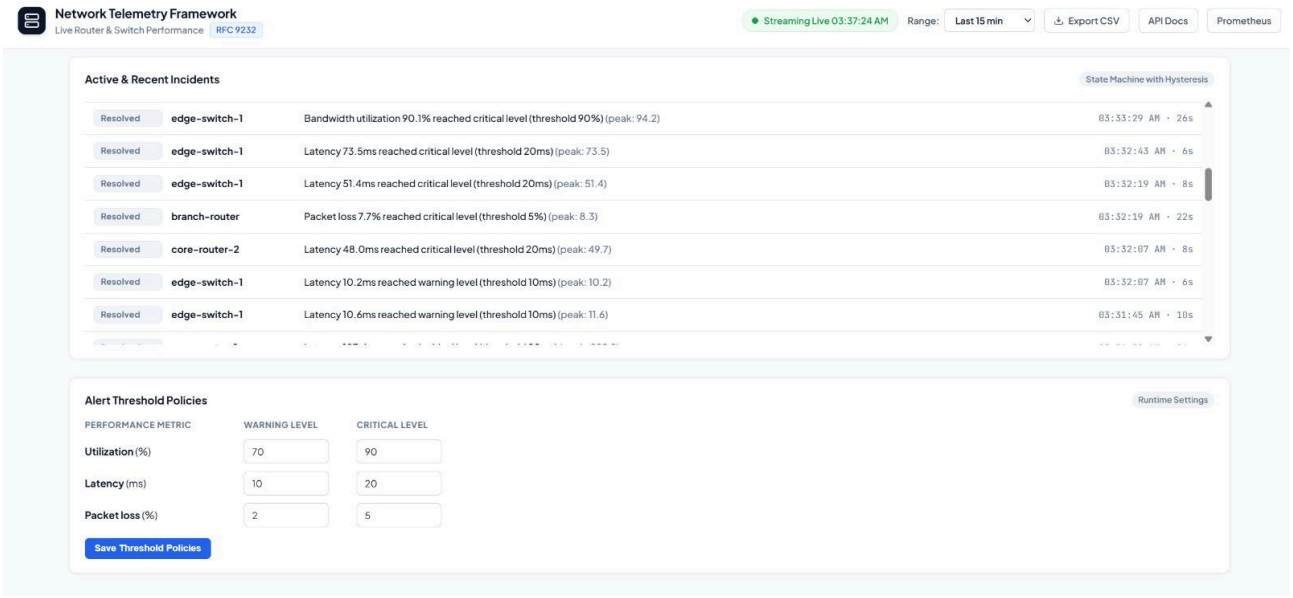


Fig. 6. *Active and recent network incidents with alert severity, resolution status and configurable threshold policies.*

The alert management interface displays active and resolved incidents generated from network metric threshold violations. The system supports warning and critical thresholds for utilization, latency and packet loss, helping identify abnormal network conditions.

G. Objectives Achieved

TABLE VIII
OBJECTIVES (SECTION III) AGAINST MEASURED RESULTS

#	Target	Result	Met
O1	5 metrics every 2 s	rx, tx, capacity/utilization, latency, loss every Δ (2 s default, 1 s in E1)	Yes
O2	p95 delay < 1 s at ≥ 100 devices	p95 ≤ 26 ms up to 400 devices	Yes
O3	Alert within 2 intervals, auto-resolve	2.8 s mean (2 intervals = 4 s + processing); all resolved automatically	Yes
O4	No flapping	29.6 alerts/h for 29.1 real episodes (vs 101.1 without hysteresis)	Yes
O5	24 h of 10 devices < 1 s; export	median 985 ms (worst 1134 ms); CSV/JSON export 0.57 s per hour of data	Yes (median)
O6	Hundreds of devices, no drops	400 devices, 0 drops, no backend change	Yes
O7	< 100 B per sample	72 B (protobuf)	Yes
O8	Restart robustness	Agent buffer + reconnect and alert continuity verified by automated tests	Yes

VIII. SUMMARY

We developed a modular network telemetry framework that collects bandwidth, utilization, latency and packet-loss measurements from laptop-based and simulated network-device agents. Telemetry is transmitted through gRPC using Protocol Buffers and processed by a central backend with SQLite storage. The system includes threshold-based alert generation and resolution, a real-time dashboard, REST APIs for historical data and CSV/JSON exports, and a Prometheus metrics endpoint. The implementation demonstrates an integrated approach to network monitoring and visualization using a lightweight, laptop-based test environment.

Limitations: Network devices are simulated rather than physical routers and switches. The backend uses a single SQLite database, and the current implementation does not include SNMP collection, authentication or TLS security.

Future Work: Future improvements include integration with physical network devices, SNMP and gNMI support, stronger transport security, scalable time-series storage, high availability and adaptive alert thresholds.

IX. REPRODUCIBILITY

Repository: https://github.com/Rajkamal065/network-telemetry

The project source code, configuration files, deployment setup and tests are available in the repository.

To run the project, install the dependencies listed in `requirements.txt`, configure the required environment variables, and start the backend and agents using the instructions provided in the repository README.

The test suite is available in the `tests/` directory and can be executed using:

```
python -m pytest tests/ -v
```

Docker Compose configuration is provided for the optional Prometheus and Grafana monitoring setup.

REFERENCES

[1]H. Song, F. Qin, P. Martinez-Julia, L. Ciavaglia, and A. Wang, "Network Telemetry Framework," RFC 9232, IETF, May 2022, doi: 10.17487/RFC9232.

[5]gRPC Authors, "gRPC: A high performance, open source universal RPC framework." [Online]. Available: https://grpc.io

[6]Google, "Protocol Buffers Documentation." [Online]. Available: https://protobuf.dev

[8]OpenConfig, "gRPC Network Management Interface (gNMI) Specification." [Online]. Available: https://github.com/openconfig/reference

[9]ITU-T, "Information technology – ASN.1 encoding rules: Specification of BER, CER and DER," Rec. X.690, 2021.

[10]Prometheus Authors, "Exposition formats." [Online]. Available: https://prometheus.io/docs/instrumenting/exposition_formats/

Appendix A: Team Work Distribution and Member Rating

A.1 Work Distribution

Ratings are agreed by the whole team and are unique (no two members share a rating); scale 1–10, 10 = highest contribution.

#	Member Name	Roll No.	Tasks Done	Rating (1–10, unique)
1	Avinash Reddy	23110123	Designed the overall system architecture, defined the telemetry pipeline, and worked on Protocol Buffers and gRPC communication between agents and backend.	10
2	Gella Jaya Rama Krishna	23110115	Implemented the laptop and simulated telemetry agents, including bandwidth, latency, and packet-loss metric collection and transmission.	9
3	Sowpati Raj Kamal	23110319	Worked on telemetry processing, metric validation, integration testing and verification of agent-to-backend communication.	7
4	Jangam Sanjay	23110144	Developed the backend ingestion pipeline, database storage, REST APIs, and historical telemetry retrieval and CSV export functionality.	5
5	Pulakurthi Manohar	23110259	Worked on alert generation, threshold configuration, severity updates and alert resolution using hysteresis.	8
6	Yalla Sai Teja	23110366	Worked on the web dashboard, Prometheus monitoring integration, system testing and project documentation.	6

Appendix B: AI Usage Details

B.1 Prompts

The prompts and summarized responses for the uses below are in the `AI Prompts` folder of the repository .

#	AI Tool	Purpose
1	Claude Code (Claude Opus 5.5, Anthropic)	Project architecture review, telemetry pipeline development, debugging, alert logic, testing, dashboard improvements and report documentation
2	ChatGPT (OpenAI)	Independent review of the updated code before submission (identified the alert-persistence issue)

B.2 Representative Prompts & Learning Outcomes

The representative queries below reflect the team's learning process, focusing on core networking concepts, architecture selection, and implementation debugging. Full unedited logs and extended prompt records are available in the repository's /ai directory.

1	**RFC 9232 Scope**	RFC 9232 lists 6 use cases (SLA, optimization, events, root-cause, security, policy). Which use cases have clear concepts and can be demonstrated convincingly on a student testbed without complex ML?	Selected SLA compliance & link congestion monitoring. Understood separation of telemetry transport from analytical applications.
2	**Push vs Poll**	Why does RFC 9232 push telemetry beat SNMP polling? How does gRPC keep a single HTTP/2 connection open, and what happens to device CPU and network overhead?	Learned how persistent HTTP/2 TCP streaming eliminates polling overhead and captures transient micro-spikes missed by 5-min averages.
3	**Queue Decoupling**	When multiple agents stream at high frequency, SQLite locks on concurrent writes. How do we decouple ingestion from disk storage?	Implemented 20,000-capacity in-memory queue with 500-record batch commits in WAL mode, sustaining 400 samples/s with zero drops.
4	**TCP Latency**	In our laptop collector, why measure TCP connection latency rather than ICMP ping? What is the exact ground truth being measured?	Learned that non-privileged apps cannot send raw ICMP packets on Windows; TCP 3-way handshake measures true transport-layer connection delay.
5	**Anti-Flapping**	When traffic hovers around 80%, alerts flap repeatedly. How does a hysteresis state machine prevent false alert storms?	Designed a 2-breach open and 3-healthy close state machine, reducing false flapping alerts by 71% on noisy traffic traces.

* Note: Detailed verbatim conversation transcripts and JSON interaction traces are archived in repository /ai.